

Modules et Packages

Le support est propriété de **Bechir Bejaoui**

1 Pourquoi modulariser un projet ?

Dans un projet réel :

- Séparation des responsabilités
- Réduction du couplage
- Réutilisabilité
- Maintenabilité
- Tests unitaires simplifiés
- Déploiement structuré

Un module est une unité logique de code. Un package est un regroupement hiérarchique de modules.

2 Module : définition concrète

Un module est simplement un fichier `.py`.

Tout au long de ce document, nous construisons progressivement une application de gestion de stock **StockPro**.

```
stockpro/  
    operations.py  
    main.py
```

operations.py

```
"""  
Module d'opérations pour la gestion de stock StockPro.  
Expose des fonctions réutilisables de calcul.  
"""  
  
def total_stock(quantite, prix_unitaire):  
    """Calcule la valeur totale d'un article en stock."""  
    return quantite * prix_unitaire  
  
def appliquer_remise(prix, taux):  
    """Applique une remise sur un prix. taux entre 0 et 1."""
```

```

        return prix * (1 - taux)

TVA = 0.20
main.py
"""
Script principal StockPro utilisant le module operations.
"""

import operations

valeur = operations.total_stock(50, 12.99)
prix_remise = operations.appliquer_remise(valeur, 0.10)

print(f"Valeur stock      : {valeur:.2f} €")
print(f"Après remise      : {prix_remise:.2f} €")
print(f"TVA appliquée       : {operations.TVA}")

```

3 Les différentes formes d'import

```

"""
Quatre formes d'import - chacune avec son cas d'usage.
"""

# Import complet - accès via namespace
import operations
print(operations.TVA)

# Import ciblé - accès direct
from operations import total_stock
print(total_stock(10, 5.99))

# Import avec alias - raccourci pratique
import operations as ops
print(ops.appliquer_remise(100, 0.15))

# Import élément avec alias
from operations import total_stock as calculer
print(calculer(20, 3.50))

# À éviter - pollution d'espace de noms
# from operations import *

```

4 Espace de noms — namespace

Chaque module possède son propre espace de noms. Deux modules peuvent définir des variables de même nom sans collision.

```
stockpro/
    entrepot_paris.py
    entrepot_lyon.py
    main.py

entrepot_paris.py
stock = 150      # unités disponibles à Paris
seuil = 20       # seuil d'alerte Paris

entrepot_lyon.py
stock = 80       # unités disponibles à Lyon
seuil = 10       # seuil d'alerte Lyon

main.py
"""
Les deux modules coexistent sans collision de noms.
"""

import entrepot_paris as paris
import entrepot_lyon as lyon

print(f"Paris : {paris.stock} unités (seuil {paris.seuil})")
print(f"Lyon : {lyon.stock} unités (seuil {lyon.seuil})")

if paris.stock < paris.seuil:
    print("Alerte stock Paris")
```

5 sys.path et résolution d'import

```
"""
Python cherche les modules dans cet ordre :
1. Répertoire courant
2. PYTHONPATH
3. Site-packages (modules installés via pip)
4. Chemins système

Afficher les chemins de recherche actifs.
"""
```

```
import sys

for chemin in sys.path:
    print(chemin)
```

6 Module exécutable vs importable

```
stockpro/
    calcul_stock.py
    rapport.py

calcul_stock.py
"""
Module StockPro : calculs de valorisation.
Peut être exécuté directement ou importé.
"""

def valoriser(quantite: int, prix: float) -> float:
    """Calcule la valeur totale d'un lot."""
    return quantite * prix

if __name__ == "__main__":
    # Exécuté uniquement si lancé directement : python calcul_stock.py
    print("Test interne :", valoriser(100, 9.99))

rapport.py
"""
Import du module calcul_stock.
Le bloc __main__ ne s'exécute pas à l'import.
"""

import calcul_stock

valeur = calcul_stock.valoriser(200, 4.50)
print(f"Valorisation du lot : {valeur:.2f} €")
```

7 Organisation en Package

Un package est un dossier contenant un fichier `__init__.py`. `__init__.py` initialise le package et définit son API publique.

```
stockpro/
```

```

__init__.py
produits.py
fournisseurs.py
commandes.py

produits.py
def ajouter_produit(nom: str, prix: float) -> dict:
    """Crée un produit StockPro."""
    return {"nom": nom, "prix": prix, "stock": 0}

***init.py
"""
Package StockPro - point d'entrée principal.

__init__.py remplit trois rôles :
1. Signale à Python que ce dossier est un package
2. Initialise le package (imports, config, hooks)
3. Définit l'API publique via __all__

Ce code s'exécute UNE SEULE FOIS au premier import du package.
"""

from .produits import ajouter_produit
from .commandes import passer_commande

__all__ = ["ajouter_produit", "passer_commande"]

print("Package StockPro initialisé")    # visible au premier import

Utilisation

from stockpro import ajouter_produit

p = ajouter_produit("Clavier", 49.99)
print(p)

```

8 all — contrôle de l'API publique

```

"""
__all__ liste les noms exportés par from package import *.
Protège les fonctions internes non destinées à l'usage externe.
"""

# stockpro/__init__.py

```

```
__all__ = ["ajouter_produit", "passer_commande"]

# _connexion_db est interne - non exporté
def _connexion_db():
    pass
```

9 Import relatif

```
"""
Le point (.) désigne le package courant.
.. désigne le package parent.
Utilisé uniquement à l'intérieur d'un package.
"""

# stockpro/commandes.py
from .produits import ajouter_produit    # même package
from .fournisseurs import get_fournisseur  # même package
```

Cas réel structuré — mini-projet StockPro

```
stockpro/
  __init__.py
  repository.py
  services.py
run.py

repository.py
"""
Accès aux données - couche repository.
"""

def get_produits() -> list:
    """Simule un accès base de données."""
    return [
        {"nom": "Clavier", "prix": 49.99, "stock": 120},
        {"nom": "Souris", "prix": 29.99, "stock": 85},
        {"nom": "Écran", "prix": 249.99, "stock": 30},
    ]

services.py
"""
```

```

Logique métier - couche services.
"""

from .repository import get_produits

def produits_en_alerte(seuil: int = 50) -> list:
    """Retourne les produits dont le stock est sous le seuil."""
    return [p for p in get_produits() if p["stock"] < seuil]

def valeur_totale_stock() -> float:
    """Calcule la valeur totale du stock."""
    return sum(p["prix"] * p["stock"] for p in get_produits())

run.py ← point d'entrée, en dehors du package
"""
Point d'entrée StockPro.
Lancé avec : python run.py
"""

from stockpro.services import produits_en_alerte, valeur_totale_stock

if __name__ == "__main__":
    print("=== Rapport StockPro ===")
    print(f"Valeur totale : {valeur_totale_stock():.2f} €")
    print("\nAlertes stock :")
    for p in produits_en_alerte(seuil=50):
        print(f"  {p['nom']} - {p['stock']} unités")

```

1 1 Rechargement d'un module

```

"""
importlib.reload recharge un module modifié sans relancer l'interpréteur.
Utile en développement interactif (notebooks, REPL).

Piège : les objets déjà importés via from module import X
gardent l'ancienne référence après reload.
"""

import importlib
import stockpro.repository as repo

# Après modification de repository.py
importlib.reload(repo)

```

```

# Attention : cette référence n'est PAS mise à jour
from stockpro.repository import get_produits
importlib.reload(repo)
# get_produits ici est toujours l'ancienne version

```

1 2 Éviter les imports circulaires

```

stockpro/
  produits.py
  commandes.py

```

Problème : `produits.py` importe `commandes.py` et inversement.

```

"""
Solution 1 : import local dans la fonction - rompt le cycle.
Solution 2 : extraire la logique commune dans un troisième module.
"""

```

```

# commandes.py - import local
def passer_commande(produit_id: int):
    from .produits import get_produit      # import différé
    produit = get_produit(produit_id)
    return {"commande": produit, "statut": "en cours"}

```

1 3 Module de configuration professionnel

```

stockpro/
  config.py
  services.py

```

`config.py`

```

"""
Configuration centralisée StockPro.
Les valeurs sont lues depuis les variables d'environnement.
"""

```

```

import os

```

```

DEBUG          = os.getenv("DEBUG", "False") == "True"
DATABASE_URL   = os.getenv("DATABASE_URL", "sqlite:///stockpro.db")

```



```

SEUIL_ALERTE = int(os.getenv("SEUIL_ALERTE", "50"))
TVA           = float(os.getenv("TVA", "0.20"))

services.py

from .config import DEBUG, SEUIL_ALERTE

def produits_en_alerte(produits: list) -> list:
    if DEBUG:
        print(f"Seuil d'alerte actif : {SEUIL_ALERTE}")
    return [p for p in produits if p["stock"] < SEUIL_ALERTE]

```

1 4 Tests internes simples

```

"""
Bloc __main__ utilisé pour tester le module isolément.
Sans framework de test - validations rapides en développement.
"""

from .repository import get_produits

def valeur_stock(produits: list) -> float:
    return sum(p["prix"] * p["stock"] for p in produits)

if __name__ == "__main__":
    produits = get_produits()
    assert len(produits) > 0, "Aucun produit"
    assert valeur_stock(produits) > 0, "Valeur nulle"
    print("Tests internes validés :", valeur_stock(produits))

```

1 5 Structuration projet professionnel

```

stockpro/
  pyproject.toml
  requirements.txt
  src/
    stockpro/
      __init__.py
      config.py
      services/
        __init__.py
        stock_service.py

```

```

    repository/
        __init__.py
        produit_repo.py
    models/
        __init__.py
        produit.py
    tests/
        test_services.py
        test_repository.py
    README.md

```

requirements.txt — dépendances à installer via `pip install -r requirements.txt`

```

requests==2.31.0
python-dotenv==1.0.0

```

pyproject.toml — standard moderne PEP 518

```

[tool.poetry]
name      = "stockpro"
version   = "1.0.0"
description = "Application de gestion de stock"

[tool.poetry.dependencies]
python    = "^3.10"
requests  = "^2.31"

```

1 6 Convention de nommage

```

"""
Règles de nommage pour les modules Python.
"""

# Correct - snake_case, descriptif, sans tiret
# gestion_stock.py
# produit_service.py
# db_connection.py
# auth_utils.py

# Incorrect
# GestionStock.py      → CamelCase réservé aux classes
# gestion-stock.py     → tiret invalide en import
# math.py              → masque le module standard
# utils.py             → trop générique

```

1 7 Bonnes pratiques

- Documenter chaque module avec une docstring en tête de fichier
 - Ne jamais exécuter de logique lourde au niveau global — encapsuler dans des fonctions
 - Centraliser la configuration dans `config.py`
 - Séparer l'accès aux données (repository) de la logique métier (services)
 - Préférer l'import absolu dans les projets structurés
 - Protéger l'API publique avec `__all__`
 - Nommer les fichiers en `snake_case` sans conflits avec les modules standards
-

1 8 Exemple avancé — Package métier complet

```
stockpro/  
  __init__.py  
  models/  
    __init__.py  
    produit.py  
  repository/  
    __init__.py  
    produit_repo.py  
  services/  
    __init__.py  
    stock_service.py  
run.py  
models/produit.py  
class Produit:  
    def __init__(self, nom: str, prix: float, stock: int):  
        self.nom = nom  
        self.prix = prix  
        self.stock = stock  
  
    def valeur(self) -> float:  
        return self.prix * self.stock
```

```

    def __repr__(self) -> str:
        return f"Produit({self.nom}, {self.prix}€, stock={self.stock})"

repository/produit_repo.py

from ..models.produit import Produit

def get_all() -> list[Produit]:
    return [
        Produit("Clavier", 49.99, 120),
        Produit("Souris", 29.99, 85),
        Produit("Écran", 249.99, 30),
    ]

services/stock_service.py

from ..repository.produit_repo import get_all

def valeur_totale() -> float:
    return sum(p.valeur() for p in get_all())

def alertes(seuil: int = 50) -> list:
    return [p for p in get_all() if p.stock < seuil]

run.py

from stockpro.services.stock_service import valeur_totale, alertes

if __name__ == "__main__":
    print(f"Valeur totale : {valeur_totale():.2f} €")
    for p in alertes(seuil=100):
        print(f"  Alerte : {p}")

```

19 Module interne vs bibliothèque externe

```

"""
Module interne : fichier .py dans votre projet.
Module externe : installé via pip, présent dans site-packages.
"""

# Vérifier l'origine d'un module installé
import requests
print(requests.__file__)    # chemin vers site-packages

# Installer une dépendance
# pip install requests

```

```
# Vérifier les modules installés
# pip list
# pip show requests
```

2 0 Synthèse

- Module = fichier `.py` — unité logique de code
 - Package = dossier avec `__init__.py` — regroupement hiérarchique
 - `__init__.py` initialise le package et définit son API publique via `__all__`
 - `__name__ == "__main__"` protège le code d'entrée à l'import
 - `sys.path` détermine où Python cherche les modules
 - Préférer l'import absolu — plus stable et plus lisible
 - Ne jamais nommer un fichier comme un module standard (`math.py`, `os.py`)
 - Séparer repository (accès données) et services (logique métier)
 - Centraliser la configuration dans `config.py` avec `os.getenv`
 - `src/` isole le code source et simule une installation réelle
-

20 Questions Intelligentes — Modules et Packages

1) Pourquoi éviter `from module import *` ?

Énoncé

Pourquoi cette forme d'import peut provoquer des bugs difficiles à détecter ?

Solution

```
stockpro/
  entrepot_paris.py
  entrepot_lyon.py
  main.py
```

```

"""
Explication :
from module import * injecte tous les noms publics
dans l'espace courant.
Risque de collision silencieuse entre modules.
"""

# entrepot_paris.py
stock = 150

# entrepot_lyon.py
stock = 80

# main.py - collision silencieuse
from entrepot_paris import *
from entrepot_lyon import *

print(stock)      # 80 - entrepot_lyon a écrasé entrepot_paris

# Correct - namespace explicite
import entrepot_paris as paris
import entrepot_lyon as lyon

print(paris.stock)  # 150
print(lyon.stock)   # 80

```

2) Pourquoi un module n'est exécuté qu'une seule fois ?

Énoncé

Que se passe-t-il lors d'un double import ?

Solution

```

stockpro/
  config.py
  main.py
"""
Explication :
Lors du premier import, Python exécute le module
et le met en cache dans sys.modules.
Les imports suivants réutilisent l'objet en cache.

```

```

"""

# config.py
print("Configuration StockPro chargée")
DEBUG = False

# main.py
import config    # affiche "Configuration StockPro chargée"
import config    # silencieux - déjà en cache

import sys
print("En cache :", "config" in sys.modules)    # True

```

3) Pourquoi éviter la logique lourde au niveau global ?

Énoncé

Quel problème provoque du code exécuté à l'import ?

Solution

```

stockpro/
  db.py
  services.py

"""
Explication :
Le code global s'exécute à chaque premier import.
Connexions, requêtes réseau → effets de bord non contrôlés.
"""

# db.py - mauvaise pratique
print("Connexion base de données...")    # exécuté à l'import

# db.py - bonne pratique
def connecter():
    print("Connexion base de données...")

if __name__ == "__main__":
    connecter()

```

4) Différence import absolu vs relatif

Énoncé

Quand utiliser l'un ou l'autre ?

Solution

```
stockpro/
  services/
    stock_service.py
  repository/
    produit_repo.py
run.py

"""
Explication :
Import absolu : chemin complet depuis la racine du projet.
→ Plus lisible, plus stable, recommandé par défaut.
Import relatif : chemin depuis le module courant avec .
→ Uniquement dans un package, pour les imports internes.
"""

# stock_service.py - import relatif (dans le package)
from ..repository.produit_repo import get_all

# run.py - import absolu (depuis l'extérieur)
from stockpro.services.stock_service import valeur_totale
```

5) Que contient sys.modules ?

Énoncé

Comment inspecter le cache des modules chargés ?

Solution

```
"""
Explication :
sys.modules est un dictionnaire.
Clé = nom du module, valeur = objet module.
Permet de vérifier qu'un module est déjà chargé.
"""

import sys
```



```
import stockpro.config as config

print("config en cache   :", "stockpro.config" in sys.modules)
print("Objet module     :", sys.modules["stockpro.config"])
print("Valeur DEBUG     :", sys.modules["stockpro.config"].DEBUG)
```

6) Pourquoi éviter les imports circulaires ?

Énoncé

Que se passe-t-il si deux modules s'importent mutuellement ?

Solution

```
stockpro/
    produits.py
    commandes.py

"""
Explication :
Python charge produits.py → qui importe commandes.py
→ qui importe produits.py avant qu'il soit initialisé
→ AttributeError ou ImportError selon le cas.

Solution : import local dans la fonction - rompt le cycle.
"""

# commandes.py
def passer_commande(produit_id: int) -> dict:
    from .produits import get_produit    # import différé
    produit = get_produit(produit_id)
    return {"produit": produit, "statut": "confirmée"}
```

7) Pourquoi utiliser __all__ ?

Énoncé

Comment protéger l'API publique d'un package ?

Solution

```
stockpro/
    __init__.py
```

```

    _interne.py
    services.py

    """
    Explication :
    __all__ contrôle ce qui est exporté par from package import *.
    Les noms absents de __all__ restent accessibles mais signalés comme internes.
    Convention : préfixer _ les fonctions internes.
    """

    # stockpro/__init__.py
    from .services import valeur_totale, alertes

    __all__ = ["valeur_totale", "alertes"]    # API publique explicite

    def _cache_interne():    # non exporté
        pass

```

8) Pourquoi éviter de nommer un fichier math.py ?

Énoncé

Quel bug subtil ce nommage peut provoquer ?

Solution

```

stockpro/
    math.py        ← problème
    calculs.py

    """
    Explication :
    Python cherche d'abord dans le répertoire courant.
    Un fichier math.py local masque le module standard math.
    """

    # math.py local - masque le module standard
    def sqrt(x):
        return x ** 0.5

    # calculs.py
    import math
    print(math.sqrt(16))    # appelle VOTRE math.py, pas le standard

```

Solution : nommer stockpro/calculs_math.py

9) Différence entre module et package namespace

Énoncé

Quel est le rôle exact de `__init__.py` ?

Solution

```
# Avec __init__.py → package standard
stockpro/
    __init__.py
    produits.py

# Sans __init__.py → namespace package (Python 3.3+)
stockpro/
    produits.py

"""
Explication :
Avec __init__.py :
- Code d'initialisation possible
- API publique via __all__
- Imports relatifs activés

Sans __init__.py :
- Namespace package - partageable entre plusieurs dossiers
- Aucune initialisation possible
- Utilisé dans les grandes architectures distribuées
"""

# Avec __init__.py
from stockpro import valeur_totale    # __init__.py expose la fonction

# Sans __init__.py
from stockpro.produits import Produit  # import direct du module
```

10) Pourquoi préférer un dossier src/ ?

Énoncé

Quel problème résout l'isolation dans src/ ?

Solution

```
# Sans src/ - risqué
stockpro/
  stockpro/
    __init__.py
  tests/
```

```
# Avec src/ - recommandé
stockpro/
  src/
    stockpro/
      __init__.py
  tests/
  pyproject.toml
```

"""

Explication :

Sans src/ : Python trouve le package stockpro dans le répertoire courant avant même qu'il soit installé + imports incohérents.

Avec src/ : le package n'est accessible que via installation (pip install -e .) + simule le comportement réel en production.

"""

```
# tests/test_services.py - avec src/
from stockpro.services.stock_service import valeur_totale

def test_valeur():
    assert valeur_totale() > 0
```

11) Rechargement — piège courant

Énoncé

Pourquoi `importlib.reload` ne met pas toujours à jour les références ?

Solution

```
"""
Explication :
reload met à jour l'objet module dans sys.modules.
Mais les références déjà extraites via from ... import
pointent toujours vers l'ancien objet.
"""

import importlib
import stockpro.repository as repo

# Après modification de repository.py
importlib.reload(repo)
print("Module rechargé :", repo.get_all())    # à jour

# Piège - référence directe non mise à jour
from stockpro.repository import get_all
importlib.reload(repo)
print("Référence directe :", get_all())      # ancienne version
```

12) python fichier.py vs python -m module

Énoncé

Pourquoi python -m est recommandé dans un package ?

Solution

```
stockpro/
  __init__.py
  services/
    stock_service.py
run.py

"""
Explication :
python services/stock_service.py
→ sys.path contient services/ → imports relatifs cassés

python -m stockpro.services.stock_service
→ sys.path contient la racine du projet → imports corrects
"""

# stock_service.py
```

```

from ..repository.produit_repo import get_all    # import relatif

if __name__ == "__main__":
    print(get_all())

# Correct
python -m stockpro.services.stock_service

# Incorrect - casse les imports relatifs
python stockpro/services/stock_service.py

```

13) Pourquoi éviter le code réseau à l'import ?

Énoncé

Quel risque en production ?

Solution

```

stockpro/
    api_client.py
    services.py

"""
Explication :
Le code au niveau global s'exécute à l'import.
Une requête réseau peut ralentir le démarrage
ou planter si le service est indisponible.
"""

# api_client.py - mauvaise pratique
import requests
reponse = requests.get("https://api.stockpro.com/produits")    # exécuté à l'import

# api_client.py - bonne pratique
import requests

def fetch_produits() -> list:
    """Requête différée - appelée explicitement."""
    reponse = requests.get("https://api.stockpro.com/produits")
    return reponse.json()

```

14) Module de configuration — pourquoi centraliser ?

Énoncé

Quel avantage dans un grand projet ?

Solution

```
stockpro/
  config.py
  services/
    stock_service.py
  repository/
    produit_repo.py

"""
Explication :
Sans config central : chaque module définit ses propres constantes
→ incohérences, modifications dispersées.

Avec config.py : un seul point de vérité.
"""

# config.py
import os

SEUIL_ALERTE = int(os.getenv("SEUIL_ALERTE", "50"))
DATABASE_URL = os.getenv("DATABASE_URL", "sqlite:///stockpro.db")
DEBUG        = os.getenv("DEBUG", "False") == "True"

# stock_service.py
from ..config import SEUIL_ALERTE

def alertes(produits: list) -> list:
    return [p for p in produits if p.stock < SEUIL_ALERTE]
```

15) Pourquoi séparer repository et services ?

Énoncé

Quel bénéfice architectural concret ?

Solution

```
stockpro/
  repository/
    produit_repo.py    ← accès données uniquement
  services/
    stock_service.py   ← logique métier uniquement

"""
Explication :
repository : sait comment récupérer les données (SQL, API, fichier).
services   : sait quoi faire avec les données (règles métier).

Bénéfices :
- Changer la source de données sans toucher la logique
- Tester services avec un repository simulé (mock)
- Lisibilité et responsabilité claire
"""

# produit_repo.py
def get_all() -> list:
    return [{"nom": "Clavier", "prix": 49.99, "stock": 120}]

# stock_service.py
from ..repository.produit_repo import get_all

def valeur_totale() -> float:
    return sum(p["prix"] * p["stock"] for p in get_all())
```

16) Module local vs bibliothèque installée

Énoncé

Comment distinguer leur origine ?

Solution

```
"""
Explication :
Module local      : chemin dans votre projet
Module installé   : chemin dans site-packages
"""

import stockpro.config as config
import requests
```



```

print("Local      :", config.__file__)      # ../stockpro/config.py
print("Installé  :", requests.__file__)    # ../site-packages/requests/__init__.py

# Lister les modules installés
# pip list
# pip show requests

```

17) Pourquoi documenter un module ?

Énoncé

Quel impact pour l'équipe ?

Solution

```

"""
Module StockPro - Gestion des alertes de stock.

Ce module expose les fonctions de surveillance du stock.
Il utilise la configuration centralisée via config.py.

Fonctions principales :
    alertes()          → produits sous le seuil d'alerte
    valeur_totale()    → valorisation globale du stock

Usage :
    from stockpro.services.stock_service import alertes
    produits_critiques = alertes(seuil=20)
"""

# Accessible via help()
import stockpro.services.stock_service as s
help(s)

```

18) Pourquoi limiter les imports au sommet du fichier ?

Énoncé

Quand faire une exception ?

Solution

```
"""
Explication :
Convention PEP 8 : tous les imports en haut du fichier.
Avantage : visibilité immédiate des dépendances.

Exceptions légitimes :
1. Import conditionnel - dépendance optionnelle
2. Import local - casser un cycle ou différer une dépendance lourde
"""

# Import conditionnel
try:
    import pandas as pd
    PANDAS_DISPONIBLE = True
except ImportError:
    PANDAS_DISPONIBLE = False

# Import local - dans la fonction
def exporter_excel(produits: list):
    import openpyxl # importé seulement si la fonction est appelée
    wb = openpyxl.Workbook()
    # ...
```

19) Namespace package sans `__init__.py`

Énoncé

Peut-on créer un package sans `__init__.py` ?

Solution

```
# Namespace package (Python 3.3+)
stockpro_plugins/
  alertes/
    email_alert.py    ← pas de __init__.py
  rapports/
    pdf_report.py     ← pas de __init__.py

"""
Explication :
Oui depuis Python 3.3 - PEP 420.
Un dossier sans __init__.py devient un namespace package.
```

```

Cas d'usage : plugins distribués dans plusieurs dossiers
qui partagent le même namespace.
Différence clé : aucun code d'initialisation possible.
"""

# Accès direct au module
from stockpro_plugins.alertes.email_alert import envoyer_alerte
from stockpro_plugins.rapports.pdf_report import generer_pdf

```

20) Architecture propre d'un projet professionnel

Énoncé

Proposer une organisation robuste pour StockPro en production.

Solution

```

stockpro/                                ← racine du projet
  pyproject.toml                          ← config packaging (PEP 518)
  requirements.txt                        ← dépendances pip
  README.md
  .env                                    ← variables d'environnement (non versionné)
  src/
    stockpro/
      __init__.py
      config.py                          ← configuration centralisée
      models/
        __init__.py
        produit.py
      repository/
        __init__.py
        produit_repo.py
      services/
        __init__.py
        stock_service.py
    tests/
      test_services.py
      test_repository.py
  run.py                                ← point d'entrée
"""

run.py - point d'entrée StockPro
Lancé avec : python run.py
"""

```

```

from stockpro.services.stock_service import valeur_totale, alertes

if __name__ == "__main__":
    print(f"Valeur totale : {valeur_totale():.2f} €")
    for p in alertes():
        print(f"  Alerte : {p}")

```

2 1 Mini-exercices de production

Exercice 1 — Prédiction

Prédire la sortie avant exécution :

```

projet/
  moduleA.py    → stock = 100 / print("A chargé")
  moduleB.py    → stock = 200 / print("B chargé")
  main.py

# main.py
from moduleA import *
from moduleB import *
print(stock)

import moduleA
import moduleA
import sys
print("moduleA en cache :", "moduleA" in sys.modules)

```

Exercice 2 — Production

Construire le package `facturation` pour `StockPro` :

```

facturation/
  __init__.py
  calculs.py
  rapport.py
run_facturation.py

```

1. `calculs.py` expose `calculer_ttc(prix_ht, taux_tva)` et `appliquer_remise(prix, taux)`
2. `rapport.py` importe depuis `calculs.py` et expose `generer_rapport(produits)`
3. `__init__.py` expose uniquement `generer_rapport` via `__all__`
4. `run_facturation.py` utilise le package avec 3 produits fictifs

```

# Votre solution ici

Solution

# facturation/calculs.py
"""
Calculs de facturation StockPro.
"""

def calculer_ttc(prix_ht: float, taux_tva: float = 0.20) -> float:
    """Calcule le prix TTC depuis un prix HT."""
    return round(prix_ht * (1 + taux_tva), 2)

def appliquer_remise(prix: float, taux: float) -> float:
    """Applique une remise. taux entre 0 et 1."""
    return round(prix * (1 - taux), 2)

# facturation/rapport.py
"""
Génération de rapports de facturation.
"""

from .calculs import calculer_ttc, appliquer_remise

def generer_rapport(produits: list) -> None:
    """Affiche un rapport TTC pour une liste de produits."""
    print("=== Rapport Facturation StockPro ===")
    total = 0
    for p in produits:
        ttc = calculer_ttc(p["prix_ht"])
        total += ttc
        print(f"  {p['nom']:20} HT: {p['prix_ht']:.2f}€  TTC: {ttc:.2f}€")
    print(f"{'Total TTC':>35} {total:.2f}€")

# facturation/__init__.py
from .rapport import generer_rapport

__all__ = ["generer_rapport"]

# run_facturation.py
from facturation import generer_rapport

produits = [
    {"nom": "Clavier", "prix_ht": 49.99},
    {"nom": "Souris", "prix_ht": 29.99},
    {"nom": "Écran", "prix_ht": 249.99},
]

```

```
if __name__ == "__main__":
    generer_rapport(produits)
```

Exercice 3 — Débogage

Ce code contient 4 erreurs. Les identifier et les corriger.

```
# main.py
from entrepot_paris import *
from entrepot_lyon import *
print(stock)      # lequel ?

# package/services.py
from package.repository import get_data    # dans le package

# math.py (fichier local)
import math
print(math.pi)    # erreur subtile

# config.py
DEBUG = os.getenv("DEBUG", "False") == "True"
```

Solution

```
# Erreur 1 : import * → collision silencieuse de stock
import entrepot_paris as paris
import entrepot_lyon as lyon
print(paris.stock)
print(lyon.stock)

# Erreur 2 : dans un package, utiliser import relatif
# package/services.py
from .repository import get_data

# Erreur 3 : math.py local masque le module standard
# Renommer math.py en calculs_stock.py
import calculs_stock
print(calculs_stock.pi)

# Erreur 4 : os non importé dans config.py
import os
DEBUG = os.getenv("DEBUG", "False") == "True"
```